The State of Simulation Frameworks for Evaluating Emerging LLM Accelerators

Stefan Maczynski
Rochester Institute of Technology
Department of Computer Engineering
Rochester, NY, USA
sxm9992@rit.edu

Amlan Ganguly
Rochester Institute of Technology
Department of Computer Engineering
Rochester, NY, USA
axgeec@rit.edu

Mark Indovina
Rochester Institute of Technology
Department of Electrical and Microelectronic Engineering
Rochester, NY, USA
maieee@rit.edu

Abstract

The increasing diversity of domain-specific AI accelerators has outpaced the tools available to simulate and analyze model execution across emerging hardware paradigms. In particular, Processing-In-Memory (PIM) architectures introduce novel data movement and execution models that are poorly captured by existing compiler backends and runtime systems. These architectures require detailed modeling of operand locality, memory access patterns, and vector-level arithmetic operations, information that is typically obscured by high-level frameworks and optimized execution paths.

This paper proposes the design of a simulation framework that retains the PyTorch front-end while introducing an intermediate profiling layer capable of capturing vector dimensions, arithmetic operations, and inter-memory data movement during both training and inference. The goal of this layer is to produce fine-grained execution traces that could serve as inputs to downstream cycle-accurate simulators, including RTL-level models and FPGA emulation environments. The proposed approach avoids translation into architecture-specific instruction sets such as CUDA or Neuron Kernel Language, enabling architecture-agnostic analysis and early-stage performance estimation.

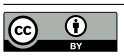
We outline the limitations of current compiler toolchains, such as TorchFX, TorchDynamo, Apache Tensor Virtual Machine (TVM), and Multi-Level Intermediate Representation (MLIR), and argue for a new instrumentation layer that bridges the abstraction gap between high-level model semantics and low-level hardware fidelity. While the implementation remains at the conceptual and prototyping stage, we illustrate how this framework could support hardware/software co-design workflows for emerging memory-centric accelerators. The aim of this work is to initiate a broader conversation around simulation methodology, architectural validation, and compiler integration for next-generation AI systems.

CCS Concepts

• **Hardware → Application specific processors**.

Keywords

Processing in Memory, Large Language Model, PyTorch



GLSVLSI '25, New Orleans, LA, USA

ACM ISBN 979-8-4007-1496-2/25/06
https://doi.org/10.1145/3716368.3735292

ACM Reference Format:
Stefan Maczynski, Amlan Ganguly, and Mark Indovina. 2025. The State of Simulation Frameworks for Evaluating Emerging LLM Accelerators. In *Great Lakes Symposium on VLSI 2025 (GLSVLSI '25), June 30–July 02, 2025, New Orleans, LA, USA.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3716368.3735292

1 Introduction

The architectural diversity of modern machine learning accelerators reflects a broadening design space in which compute is increasingly co-optimized with memory and data movement patterns. Architectures such as NVIDIA's GPUs [18], Google's TPUs [9], Amazon's Trainium [8], and emerging Processing-In-Memory (PIM) platforms [27, 29] implement fundamentally different strategies for operand locality, parallelism, and compute granularity in deep neural network (DNN) and Transformer workloads. Yet, despite these divergences, the dominant abstraction layer for model development remains PyTorch [20], which provides an imperative and expressive interface for model authorship and experimentation.

This architectural heterogeneity introduces a critical challenge for accelerator research and hardware/software co-design: how to accurately represent and analyze model execution behavior across radically different hardware paradigms, particularly during early-stage design or pre-silicon validation. PIM, in particular, places an unusual emphasis on minimizing data movement rather than maximizing arithmetic throughput, making it essential to observe operand locality, inter-memory traffic, and fine-grained compute structure at a level of fidelity that most ML frameworks and compilers do not expose. Traditional metrics such as FLOPs or latency, while sufficient for GPU throughput benchmarking, obscure the memory-centric bottlenecks of data-centric architectures like PIM.

Accurate simulation of such architectures, particularly for cycle-level validation via HDL or FPGA emulation, requires precise visibility into:

- Instruction issuance timing
- Operand vector sizes and shapes
- Memory hierarchy interactions
- Data movement across internal buffers and memory banks

Existing tools within the PyTorch ecosystem, including TorchDynamo [21], TorchFX [23], and LazyTensor [26], support varying degrees of intermediate representation (IR) capture, graph transformation, and execution deferral. These components have enabled the rise of new compiler backends such as TorchInductor [21], TVM [3], and MLIR-based toolchains [15]. However, these compiler stacks are explicitly designed for runtime performance and kernel fusion

on known targets, and are not intended to capture or expose the underlying microarchitectural behaviors that shape performance on nonstandard or experimental architectures. Furthermore, backends such as CUDA [18] and XLA [25] are tightly coupled to proprietary runtime systems, making them unsuitable for simulating or validating architectural hypotheses in a portable, backend-agnostic way.

In this paper, we propose a simulation-oriented execution framework that operates as an intermediate instrumentation layer within the PyTorch stack. Rather than compiling models to a vendor-specific backend, this layer non-invasively captures model execution at the point of tensor operation, collecting detailed information about arithmetic patterns, tensor dimensions, and inter-memory data movement. These traces serve as inputs for cycle-accurate simulators targeting custom accelerators, with particular emphasis on PIM systems, where energy and performance are dominated not by compute alone, but by the orchestration of memory flows and in-situ data reuse.

Our proposed design leverages TorchDynamo to trace execution paths dynamically, and TorchFX to construct a graph-level intermediate representation suitable for analysis, transformation, or downstream simulation. Optional integration with LazyTensor or MLIR allows for higher-level graph optimization or backend lowering when needed. This architecture is not a performance optimizer or compiler backend, but a diagnostic and simulation-enabling tool that operates between the framework and the hardware. We believe that such a profiling-oriented, backend-agnostic simulation layer is essential to enabling architectural experimentation, functional verification, and trace-based modeling for next-generation accelerators.

2 Current Execution and Compilation Toolchains

The execution of machine learning workloads has shifted dramatically in recent years, driven by the emergence of specialized compiler infrastructures, graph rewriting systems, and hardware-specific runtime backends, shown in Figure 1. These advances have enabled powerful optimizations such as operator fusion, memory layout transformation, quantization, and device-specific code generation. Frameworks such as PyTorch have evolved to support a range of execution paths, each of which allows different levels of visibility into model structure and execution behavior. These backends are now commonly used to target optimizing compilers such as TorchInductor, TVM, and MLIR-based toolchains [3, 15, 20], which in turn generate low-level code for specific hardware targets, including CUDA [18] and XLA [25].

However, these tools are predominantly designed with a focus on runtime performance and deployment stability, and do not expose the granularity needed to support cycle-level performance modeling or architectural simulation. This limitation is particularly problematic for novel and memory-centric designs such as Processing-In-Memory (PIM), where the performance bottlenecks are often dominated not by arithmetic throughput but by operand locality, interconnect contention, and memory hierarchy behavior. Furthermore, the diversity of backend instruction sets and runtime systems introduces substantial heterogeneity, making it difficult to conduct cross-architecture or cross-stack comparisons in a consistent or backend-agnostic manner.

2.1 PyTorch Execution Backends

PyTorch has emerged as the dominant front-end framework for deep learning research and deployment, owing to its expressive imperative programming model and its ecosystem of extensible intermediate representations (IRs) and backend compilation tools. To accommodate both research flexibility and production-level performance, PyTorch has evolved several mechanisms for tracing, transforming, and compiling models. These execution backends form the foundation of modern PyTorch compilation stacks but fall short when applied to high-fidelity architectural simulation.

TorchDynamo [21] is a Python-level bytecode tracer that intercepts tensor operations during program execution and rewrites them into FX-style IR graphs. It enables dynamic capture of control flow and integrates with downstream compilers such as TorchInductor and TensorRT. While TorchDynamo excels in JIT graph extraction and backend routing, it does not expose detailed execution metadata such as per-op latency, data movement volume, or memory hierarchy interactions, characteristics that are critical for hardware simulation or energy modeling.

TorchFX [23] provides a Python-native symbolic tracer that converts nn.Module instances into a graph-based IR (GraphModule) composed of statically analyzable operator nodes. FX is widely used for graph transformation, operator fusion, and quantization-aware training workflows. However, it only captures tensor operation flows and lacks native support for modeling hardware-level concerns such as operand locality, memory address mapping, or vector width dependencies. While it offers a clean entry point for instrumentation, its static nature limits the insight it can provide into runtime behavior without external profiling integration.

TorchScript [22], introduced as PyTorch's first major intermediate format, enables ahead-of-time (AOT) compilation by transforming annotated models into a static, typed IR suitable for C++ execution. Though it supports more Python control flow than FX, it imposes significant syntactic restrictions on dynamic code and lacks extensibility for introspection. TorchScript is primarily geared toward deployment and inference optimization, making it difficult to use as a simulation or architectural analysis tool, especially for emerging architectures with nonstandard execution semantics.

LazyTensor [26] is a backend infrastructure that defers tensor operation execution to construct a global computation graph, enabling whole-graph optimization and backend lowering. Used internally by PyTorch/XLA and TorchInductor, LazyTensor supports efficient fusion and memory reuse optimizations across accelerators such as TPUs and GPUs. However, LazyTensor abstracts away immediate tensor materialization, making it challenging to recover fine-grained information such as precise operand lifetimes, memory residency, and execution timing, details which are essential for validating hardware design hypotheses or simulating cycle-level behavior in Processing-In-Memory (PIM) systems.

Together, these backends provide powerful abstractions for graph-level optimization and code generation, but they are optimized for performance and code portability, not architectural fidelity. None provides a standardized mechanism for tracking arithmetic intensity, vector sizes, or inter-memory data movement as required for

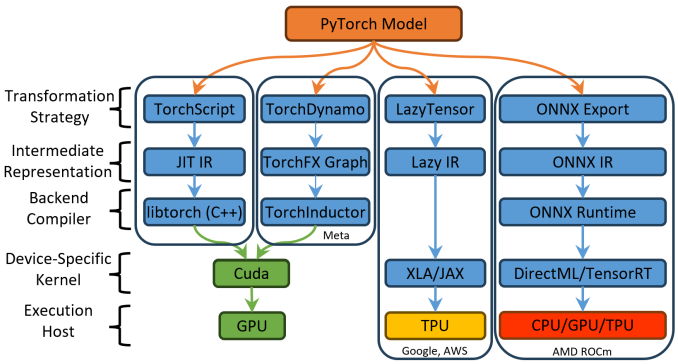


Figure 1: PyTorch Backend Compilation and Execution Stack

evaluating custom architectures. These limitations motivate the need for an intermediate simulation-oriented execution layer that can observe PyTorch programs at the point of operation and collect rich traces suitable for architectural validation.

2.2 Compiler Toolchains and Intermediate Representations

In addition to PyTorch-native intermediate representations (IRs), the machine learning ecosystem includes a variety of compiler toolchains and IRs designed to optimize model execution across a range of hardware targets. These toolchains serve as bridges between high-level frameworks and device-specific code generation, providing opportunities for graph-level optimizations, fusion, and memory planning. However, their emphasis on runtime performance and deployment portability makes them less suited to tasks such as hardware-level introspection, operand-level profiling, and simulation fidelity, especially for novel accelerator designs without existing compilation targets.

Apache TVM (Tensor Virtual Machine) [3] is an open-source deep learning compiler stack designed to optimize model execution across a wide range of hardware platforms, including CPUs, GPUs, mobile accelerators, and custom ASICs. TVM transforms high-level models from front-ends like PyTorch, TensorFlow, and ONNX into its own intermediate representations, most notably the Relay IR, a statically typed, functional language for representing neural network computations. After Relay-level optimizations, TVM applies a series of lowering passes that progressively transform the model into tensorized loop nests and memory-aware low-level IR, enabling fine control over layout, tiling, and memory scope (e.g., global, shared, local). These transformations are guided by either manually defined schedules or an auto-scheduler that searches the schedule space using machine learning-based cost models.

While TVM exposes detailed control over low-level execution through its scheduling language and memory annotations, its tooling is oriented toward runtime optimization rather than architectural simulation. It does not natively track cycle-accurate timing, memory port contention, or per-cycle operand movement, features essential for evaluating hardware proposals like Processing-In-Memory (PIM), where compute and memory are tightly integrated. Furthermore, using TVM to model novel hardware requires writing custom backends and target-specific lowering rules, which introduces significant complexity and may abstract away hardware-level behaviors critical for early-stage design exploration. As such, while TVM is powerful for code generation and performance tuning, it is not ideally suited for tracing and analyzing the hardware-level data dynamics required by cycle-accurate simulators or pre-silicon validation frameworks.

MLIR (Multi-Level Intermediate Representation) [15] is a compiler infrastructure developed as part of the LLVM ecosystem to support the construction of modular and extensible compiler toolchains. MLIR introduces the concept of dialects, domain-specific intermediate representations that can model computation at multiple abstraction levels, from high-level tensor operations to low-level memory semantics. It provides a unified infrastructure for transformations, analyses, and backend lowering across these levels, and is already used as the foundation for compiler frontends such as XLA's mhlo and PyTorch's torch-mlir.

In principle, MLIR could be used to build a simulation-oriented dialect that explicitly models PIM-style execution, including operand locality, memory transfers, and array-level parallelism. However, doing so requires considerable engineering effort: defining dialects, implementing transformation passes, and constructing hardware-aware cost models. While MLIR is highly flexible, this complexity limits its accessibility for early-stage architectural studies or lightweight simulation workflows that need to integrate directly with PyTorch.

XLA [25] (Accelerated Linear Algebra) is a domain-specific compiler infrastructure originally developed to optimize TensorFlow and later extended for JAX [7]. XLA lowers model computations into a specialized intermediate representation called HLO (High-Level Optimizer), which enables aggressive operator fusion, algebraic simplification, and backend-specific kernel generation. XLA's compilation pipeline includes multi-pass optimizations such as layout assignment, instruction fusion, and buffer assignment, and it targets high-throughput execution on accelerators like GPUs and TPUs.

In conjunction with JAX, XLA supports partial evaluation and automatic differentiation by tracing pure and statically composed (PSC) subroutines written in Python. This design allows highly performant code generation for key numerical routines, while leaving the orchestration logic in Python. However, this model imposes strong assumptions about static graph structure and pure functions, limiting its applicability to architectures with irregular or stateful behavior. Moreover, XLA's backend abstractions are tightly coupled with specific runtime environments and rely on opaque kernel primitives (e.g., cuDNN or cuRAND), which restrict introspection into memory hierarchy, operand locality, or cycle-level timing. These limitations make XLA a powerful tool for production optimization, but not a suitable foundation for simulation-based hardware co-design or evaluation of novel execution models such as Processing-In-Memory.

ONNX (Open Neural Network Exchange) [19] provides a standardized IR for model interchange across frameworks, including PyTorch and TensorFlow [1]. ONNX graphs support static shape inference and limited transformation via tools like ONNX Runtime. While ONNX is valuable for model portability and backend interoperability, it lacks semantic depth and runtime instrumentation. Control flow is flattened or discarded during export, and ONNX models are effectively declarative graphs, with no mechanism to annotate or observe dynamic behavior, memory access,

or operand reuse, making ONNX inappropriate as a substrate for architecture-aware simulation.

In summary, the compiler toolchains and IRs described above provide useful abstractions for optimizing model execution on established hardware targets. However, their primary focus on runtime performance, deployment portability, or model interchange limits their suitability for fine-grained architectural analysis. None of these systems natively expose the temporal, spatial, or memory-residency characteristics of operand execution that are critical for evaluating architectural innovations, particularly in non-von Neumann paradigms such as Processing-In-Memory (PIM). Moreover, while frameworks like MLIR offer the flexibility to model such behaviors, the engineering overhead required to do so remains prohibitive for early-stage experimentation. These limitations highlight the need for a lightweight, PyTorch-compatible intermediate execution layer capable of logging fine-grained tensor activity, operand movement, and arithmetic structure, enabling trace-driven simulation without dependence on hardware-specific compilers or runtime constraints.

3 Hardware Simulation Tools

Cycle-accurate simulators play a crucial role in architectural design and evaluation, particularly for CPUs and GPUs. Tools like gem5 [17], GPGPU-Sim [28], and Accel-Sim [11] offer detailed timing and performance modeling for conventional processor architectures. gem5 provides a modular infrastructure for simulating out-of-order and in-order processors, memory hierarchies, and devices at the cycle level. It supports full system simulation, including booting unmodified operating systems, and is widely used for CPU-centric research and memory system studies. However, while gem5 is powerful for general-purpose CPU modeling, it lacks native support for simulating non-von Neumann architectures like Processing-In-Memory (PIM), where computation occurs near or inside the memory array and does not follow the classical instruction-fetch-execute model.

GPGPU-Sim is a well-known CUDA-compatible GPU simulator capable of simulating NVIDIA's virtual ISA (PTX) with high fidelity. Recent work has improved its performance through parallelization and enhanced model accuracy [28], but the simulator remains tightly coupled to conventional SIMT execution models and CUDA-style memory hierarchies. Accel-Sim, a successor and extension to GPGPU-Sim, introduces both execution- and trace-driven simulation modes, supporting native machine ISA traces (SASS) extracted via instrumentation tools like NVBit. It significantly improves fidelity for simulating workloads using closed-source, hand-tuned libraries like cuDNN, and supports modern GPU architectures including Kepler, Pascal, Volta, and Turing [11]. However, Accel-Sim, like its predecessors, is specialized for GPU-style throughput architectures and does not generalize well to memory-centric models like PIM, where timing behavior, operand locality, and data-dependent control flow require customized instrumentation and simulation infrastructure.

FPGA-based frameworks such as FireSim [10] provide high-fidelity emulation of RISC-V and other hardware platforms based on synthesizable RTL. These platforms are valuable for validating architectural prototypes, but their reliance on RTL-level input makes them difficult to drive using high-level ML frameworks. Feeding PyTorch workloads into these platforms requires the generation of cycle-accurate traces of arithmetic and memory behavior, information not provided by PyTorch backends or existing compilers.

At the lowest level, simulation of Verilog or SystemVerilog designs using HDL simulators provides the most accurate validation path, particularly for PIM prototypes. These simulators require carefully structured per-cycle inputs, including tensor shapes, memory access patterns, and interconnect scheduling. However, no existing toolchain can generate such traces automatically from high-level model descriptions, creating a bottleneck between ML development and architectural exploration.

Collectively, these tools form a rich ecosystem for CPU and GPU simulation, but they fall short of supporting architecture-aware analysis or simulation of novel, reconfigurable, or in-memory compute platforms. The inability to extract fine-grained execution semantics from ML frameworks like PyTorch and to translate those semantics into trace formats consumable by hardware simulation environments motivates the development of an intermediate instrumentation layer that can bridge this critical gap.

4 Memory Hierarchy Modeling and Operand Movement

In PIM architectures, execution performance is often constrained more by data movement and memory hierarchy timing than by raw compute throughput. Modeling operand locality, bank conflicts, and DRAM access delays is therefore essential. Simulators such as PIM-Sys [5] and PIMCoSim [24] explicitly model memory-side operand routing and subarray-level behavior, highlighting the impact of row buffer conflicts and internal contention.

Architectures like TransPIM [29] demonstrate this need in practice: although not a simulator itself, TransPIM was evaluated using a custom simulation stack built on Ramulator [12], allowing the authors to back-annotate realistic DRAM latency effects into their PIM datapaths. This methodology enabled them to capture the impact of memory timing on attention layers and feed-forward blocks in large transformer workloads.

The simulation framework proposed in this work supports a similar goal. By extracting detailed tensor traces from PyTorch and allowing back-annotation of memory timing through DRAM simulators, it facilitates cycle-aware modeling of operand movement.

5 Design of Simulation Framework

The proposed simulation framework functions as a non-intrusive instrumentation layer embedded within the PyTorch execution path, shown in Figure 2. Rather than compiling to hardware-specific instruction sets like CUDA [18] or XLA [25], it captures tensor-level execution semantics directly during model runtime, preserving dynamic behavior while exposing the information necessary for architectural simulation.

This framework integrates with PyTorch's intermediate representations, including TorchDynamo [21] for dynamic graph tracing, TorchFX [23] for symbolic graph construction and transformation, and optionally MLIR [15] for extensibility through custom dialects. These components allow the framework to extract a structured trace of model execution, encompassing operation types, tensor dimensions, and memory interactions, without requiring code modification or static compilation.

The simulation trace generated reflects not only the computational structure of the model but also its runtime dataflow behavior. This enables back-annotation of timing information using DRAM simulators such as Ramulator [12], as employed in prior architecture studies like TransPIM [29]. Through such integration, the framework supports timing-aware simulation of operand locality, memory access latency, and potential contention across subarrays and banks.

By operating at the IR level and maintaining compatibility with PyTorch's eager execution semantics, the framework enables rapid evaluation of emerging models and quantization techniques. For example, it supports the analysis of Llama Guard 3-1B-INT4 [6] and AWQ [16] without requiring operator rewriting or backend recompilation. Model execution can be profiled directly from PyTorch, making the system particularly well-suited for co-design workflows that require fast iteration across both network architectures and hardware configurations.

This design bridges the gap between high-level ML development and low-level architectural validation, enabling simulation workflows tailored for memory-centric accelerators where operand movement, rather than arithmetic throughput, often determines performance.

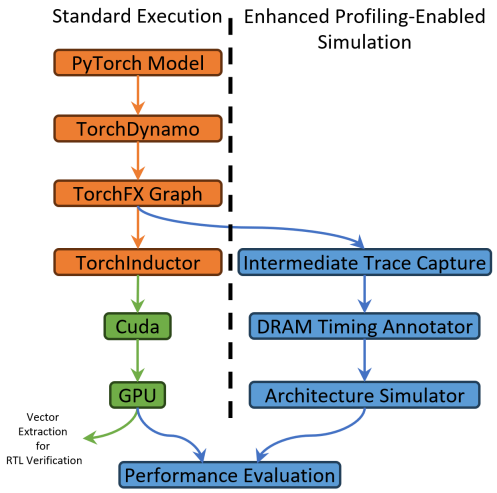


Figure 2: Comparison Between Standard Execution and Profiling-Enabled Simulation Flow

6 Hardware/Software Co-Design for PIM Architectures

While significant progress has been made in profiling and optimizing transformer workloads for GPUs and edge AI devices, similar co-design methodologies do not yet exist for Processing-In-Memory (PIM) architectures. GPU profiling techniques, such as those detailed in [4], rely on assumptions about centralized memory hierarchies, kernel fusion, and bandwidth-aware scheduling that do not translate to PIM's decentralized, locality-driven execution model. Metrics like FLOPs and kernel latency are often misleading in contexts where data movement dominates energy and performance.

Likewise, automated hardware/software co-design flows for edge AI [2] enable joint optimization of model and accelerator but depend on pre-defined execution models and backend-specific compilers. These approaches cannot evaluate architectural innovations like in-memory arithmetic, sense-amplifier-level execution, or subarray-parallel compute.

Currently, there is no tooling infrastructure that allows developers to rapidly iterate across both hardware configurations and model architectures, particularly for evaluating memory access patterns, operand scheduling, or the impact of sparsity and quantization on emerging compute fabrics. The absence of such a framework limits the design space exploration needed for PIM.

The proposed intermediate execution layer directly addresses this limitation by enabling fine-grained profiling of tensor-level behavior without reliance on specific instruction sets or vendor runtimes. It supports simulation-ready trace generation for a variety of models, and opens the door to iterative, architecture-agnostic co-design in emerging memory-centric accelerators.

7 Prototype Results

Although the full intermediate execution layer proposed in this work has not yet been realized, we have developed a functional prototype that demonstrates the feasibility of extracting meaningful execution metadata from PyTorch-based models for simulation purposes. To explore this, we utilized the PyTorch implementation of OpenNMT [13, 14], a widely adopted neural machine translation toolkit that supports Transformer-based architectures and has been optimized for both research and production environments.

This prototype implementation focuses on capturing execution behavior during inference using the CTranslate2 runtime, which provides an optimized backend for Transformer models exported from OpenNMT-py. Through manual instrumentation and selective logging within the model's forward pass, we were able to extract tensor shapes, dataflow patterns, and layer-level arithmetic activity. These traces, while currently unstructured, reflect the underlying operational characteristics of key Transformer components such as attention, softmax, and feed-forward layers.

This early effort has enabled us to visualize operand flow, dump intermediate tensor data, and analyze compute and memory behavior during inference. These preliminary traces offer early insight into how a PIM-based hardware architecture would need to orchestrate a Transformer network, particularly in terms of operand reuse, memory movement, and potential vectorization bottlenecks. In addition, the prototype supports the extraction of real functional test vectors, which can be applied directly in RTL simulations, allowing design verification to extend beyond synthetic benchmarks and better reflect actual model behavior. While the prototype lacks integration with TorchDynamo or TorchFX and does not yet produce cycle-accurate traces, it confirms that such instrumentation is feasible using standard PyTorch execution paths and can be performed without modifying the model architecture or training workflow.

Importantly, this prototype supports the core thesis of this work: that a simulation-oriented instrumentation layer can be constructed using PyTorch's existing IR and backend hooks, and that such a layer can support realistic, architecture-aware simulation of emerging models. We believe these initial results motivate continued development of the proposed utility, and demonstrate the technical viability of capturing high-level model execution for backend-agnostic hardware simulation.

8 Conclusions

This work presents a conceptual design for a simulation-oriented execution layer that captures fine-grained model behavior within the PyTorch framework, enabling high-fidelity trace generation for cycle-accurate simulation of novel AI hardware, particularly Processing-In-Memory (PIM) architectures. By leveraging existing intermediate representations, TorchDynamo for dynamic trace capture, TorchFX for graph-level analysis, and optionally MLIR for extensibility, this proposed layer operates without binding to any specific instruction set or vendor runtime, thereby remaining agnostic to backend assumptions.

Unlike traditional compiler toolchains that prioritize runtime efficiency or hardware-specific optimization, the proposed framework is explicitly designed to instrument PyTorch models in situ, logging tensor dimensions, arithmetic patterns, and inter-memory data transfers. These traces can then serve as direct inputs to hardware-level simulators, from RTL environments to FPGA-based prototypes, enabling realistic validation of architectural hypotheses.

This approach also facilitates rapid evaluation of emerging model innovations, such as the Llama Guard 3-1B-INT4 quantized safeguard model [6] and AWQ's activation-aware weight quantization method [16]. Because the proposed simulation layer operates directly on PyTorch execution, new models can be evaluated in short order without requiring recompilation, reimplementation, or custom operator definitions.

By providing a bridge between model authorship and architectural simulation, this work aims to accelerate the hardware/software co-design process and enable performance modeling of architectures that fall outside the scope of existing runtime and compilation infrastructures. Future work includes implementation of the intermediate profiling layer, integration with existing hardware simulators, and exploration of memory hierarchy and operand movement modeling tailored for non-von Neumann compute paradigms.

References

[1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, et al. 2016. {TensorFlow}: a system for {Large-Scale} machine learning. In *12th USENIX symposium on operating systems design and implementation (OSDI 16)*. 265–283.

[2] Oliver Bringmann, Wolfgang Ecker, Ingo Feldner, Adrian Frischknecht, Christoph Gerum, Timo Hämäläinen, Muhammad Abdullah Hanif, Michael J Klaiber, Daniel Mueller-Gritschneder, Paul Palomero Bernardo, et al. 2021. Automated HW/SW co-design for edge AI: State, challenges and steps ahead. In *Proceedings of the 2021 International Conference on Hardware/Software Codesign and System Synthesis*. 11–20.

[3] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.

[4] Scott Cheng, Jun-Liang Lin, Murali Emani, Siddhisanket Raskar, Sam Foreman, Zhen Xie, Venkatram Vishwanath, and Mahmut Taylan Kandemir. 2024. Thorough characterization and analysis of large transformer model training at-scale. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8, 1 (2024), 1–25.

[5] Derek Christ, Lukas Steiner, Matthias Jung, and Norbert Wehn. 2024. PIMSys: A Virtual Prototype for Processing in Memory. In *Proceedings of the International Symposium on Memory Systems*. 26–33.

[6] Igor Fedorov, Kate Plawiak, Lemeng Wu, Tarek Elgamal, Naveen Suda, Eric Smith, Hongyuan Zhan, Jianfeng Chi, Yuriy Hulovatyy, Kimish Patel, et al. 2024. Llama Guard 3-1B-INT4: Compact and Efficient Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2411.17713* (2024).

[7] Roy Frostig, Matthew James Johnson, and Chris Leary. 2018. Compiling machine learning programs via high-level tracing. *Systems for Machine Learning* 4, 9 (2018).

[8] Xinwei Fu, Zhen Zhang, Haozheng Fan, Guangtai Huang, Mohammad El-Shabani, Randy Huang, Rahul Solanki, Fei Wu, Ron Diamant, and Yida Wang. 2024. Distributed training of large language models on aws trainium. In *Proceedings of the 2024 ACM Symposium on Cloud Computing*. 961–976.

[9] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. 2023. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*. 1–14.

[10] Sagar Karandikar, Howard Mao, Donggyu Kim, David Biancolin, Alon Amid, Dayeol Lee, Nathan Pemberton, Emmanuel Amaro, Colin Schmidt, Aditya Chopra, Qijing Huang, Kyle Kovacs, Borivoje Nikolic, Randy Katz, Jonathan Bachrach, and Krste Asanović. 2018. FireSim: FPGA-accelerated Cycle-exact Scale-out System Simulation in the Public Cloud. In *Proceedings of the 45th Annual International Symposium on Computer Architecture* (Los Angeles, California) *(ISCA '18)*. IEEE Press, Piscataway, NJ, USA, 29–42. doi:10.1109/ISCA.2018.00014

[11] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 473–486. doi:10.1109/ISCA45697.2020.00047

[12] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A fast and extensible DRAM simulator. *IEEE Computer architecture letters* 15, 1 (2015), 45–49.

[13] Guillaume Klein, François Hernandez, Vincent Nguyen, and Jean Senellart. 2020. The OpenNMT neural machine translation toolkit: 2020 edition. In *Proceedings of the 14th Conference of the Association for Machine Translation in the Americas (Volume 1: Research Track)*. 102–109.

[14] Guillaume Klein, Yoon Kim, Yuntian Deng, Jean Senellart, and Alexander Rush. 2017. OpenNMT: Open-Source Toolkit for Neural Machine Translation. In *Proceedings of ACL 2017, System Demonstrations*, Mohit Bansal and Heng Ji (Eds.). Association for Computational Linguistics, Vancouver, Canada, 67–72. https://aclanthology.org/P17-4012/

[15] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. 2020. MLIR: A compiler infrastructure for the end of Moore's law. *arXiv preprint arXiv:2002.11054* (2020).

[16] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.

[17] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, et al. 2020. The gem5 simulator: Version 20.0+. *arXiv preprint arXiv:2007.03152* (2020).

[18] Nvidia. 2007. CUDA. https://docs.nvidia.com/cuda/

[19] ONNX. 2019. ONNX. https://github.com/onnx/onnx.git

[20] A Paszke. 2019. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703* (2019).

[21] pytorch. 2024. Torch Dynamo. https://pytorch.org/docs/stable/torch.compiler_dynamo_overview.html

[22] PyTorch. 2024. TorchScript. https://huggingface.co/docs/transformers/en/torchscript

[23] James Reed, Zachary DeVito, Horace He, Ansley Ussery, and Jason Ansel. 2022. torch. fx: Practical program capture and transformation for deep learning in python. *Proceedings of Machine Learning and Systems* 4 (2022), 638–651.

[24] Jinyoung Shin, Seongmo An, Sangho Lee, and Seung Eun Lee. 2024. PIMCoSim: Hardware/Software Co-Simulator for Exploring Processing-in-Memory Architectures. *Electronics* 13, 23 (2024). doi:10.3390/electronics13234795

[25] Daniel Snider and Ruofan Liang. 2023. Operator fusion in XLA: analysis and evaluation. *arXiv preprint arXiv:2301.13062* (2023).

[26] Alex Suhan, Davide Libenzi, Ailing Zhang, Parker Schuh, Brennan Saeta, Jie Young Sohn, and Denys Shabalin. 2021. LazyTensor: combining eager execution with domain-specific compilers. *arXiv preprint arXiv:2102.13267* (2021).

[27] Purab Ranjan Sutradhar, Sathwika Bavikadi, Sai Manoj Pudukotai Dinakarrao, Mark A. Indovina, and Amlan Ganguly. 2024. 3DL-PIM: A Look-Up Table Oriented Programmable Processing in Memory Architecture Based on the 3-D Stacked Memory for Data-Intensive Applications. *IEEE Transactions on Emerging Topics in Computing* 12, 1 (2024), 60–72. doi:10.1109/TETC.2023.3293140

[28] Jun Wang and Jiaquan Gao. 2020. Parallelizing GPGPU-Sim for Faster Simulation with High Fidelity. *IEEE Design & Test* 37, 4 (2020), 83–91. doi:10.1109/MDAT.2020.2986738

[29] Minxuan Zhou, Weihong Xu, Jaeyoung Kang, and Tajana Rosing. 2022. TransPIM: A memory-based acceleration via software-hardware co-design for transformer. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1071–1085.